

DATA SCIENCE & MACHINE LEARNING MODULE II

- **INTRODUCTION TO MACHINE LEARNING**
- **HOW MACHINES LEARN -DATA STORAGE, ABSTRACTION, GENERALISATION, EVALUATION**
- **TYPES OF MACHINE LEARNING ALGORITHMS**
- **K-NEAREST NEIGHBOUR ALGORITHM**
- **PROBABILISTIC LEARNING**
- **NAIVE BAYES CLASSIFIER ALGORITHM**
- **THE LAPLACE ESTIMATOR**

INTRODUCTION TO MACHINE LEARNING

Machine Learning (ML) is a subset of Artificial Intelligence (AI) that enables systems to **learn from data** and **improve performance** over time without being explicitly programmed. It focuses on creating algorithms that can **identify patterns**, **make decisions**, and **predict outcomes**.

Instead of programming the computer with rules (like traditional programming), we **give it data**, and it **learns patterns** from the data to make predictions.

Example:

Imagine you want to teach a computer to recognize whether a student will **pass or fail**.

- You collect data: attendance, test scores, study hours.
- You give this data to a machine learning algorithm.
- The computer finds patterns (like: students with attendance > 80% usually pass).
- Later, for a **new student**, it can predict: **Pass or Fail** based on learned patterns.

HOW MACHINES LEARN

Machines learn by going through a cycle of understanding and improving based on data.

1. Data Storage

This is the first step where the machine collects and stores raw data from various sources like sensors, websites, CSV files, or databases.

Example:

A machine learning model is trained to predict student performance.

Stored data includes:

- Hours of study
- Attendance
- Internal marks
- Final result (Pass/Fail)

Keep the data safely so it can be used for learning.

2. Abstraction

The machine tries to find patterns or relationships in the data by simplifying complex information.

How:

- Removes unnecessary details
- Converts raw data into useful features
- Focuses only on what is important to learn

Example:

From student data, it may identify that study hours and attendance are more important than hobbies for predicting performance.

Extract only the important features from data.

3. Generalization

The machine learns from training data and tries to make predictions on new, unseen data.

Good models don't memorize — they generalize!

They should work well on real-world data, not just training data.

Example:

If the model has learned that “students with more than 75% attendance usually pass,” it should be able to apply that rule to a new student.

Apply what it learned to new situations.

4. Evaluation

Check how well the model is performing. The model is tested using test data (data it has never seen before).

Metrics used:

- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix

If we test the model on 100 students and it predicts 90 correctly, the accuracy is 90%.

Measure and improve model performance.

TYPES OF MACHINE LEARNING ALGORITHMS

There are **four main types** of Machine Learning based on how machines learn from data:

1. Supervised Learning

The model learns from labeled data (input and output are known).

Predict the output from new inputs.

Examples:

- Email spam detection (spam or not spam)
- Weather prediction (rain or no rain)

Algorithms:

- Linear Regression
- Logistic Regression
- Decision Trees
- Support Vector Machines (SVM)
- k-Nearest Neighbors (KNN)

Algorithm	Explanation
Linear Regression	Predicts a continuous value based on a straight-line relationship between input and output. Example: Predicting house price based on size.
Logistic Regression	Used for binary classification (Yes/No). Example: Will a customer buy or not?
Decision Tree	Splits data based on decision rules (like a flowchart). Example: Classifying if a loan should be approved.
k-Nearest Neighbors (KNN)	Finds the closest data points to make a prediction. Example: Classify animals based on size and weight.
Support Vector Machine (SVM)	Finds the best boundary that separates classes. Example: Handwritten digit recognition.

2. Unsupervised Learning

The model learns from unlabeled data (only input is given, no output).

Find hidden patterns or groupings in the data.

Examples:

- Customer segmentation
- Market basket analysis

Algorithms:

- K-Means Clustering
- Hierarchical Clustering
- Principal Component Analysis (PCA)

Algorithm	Explanation
K-Means Clustering	Groups data into 'k' clusters based on similarity. Example: Segmenting customers into groups.
Hierarchical Clustering	Builds a tree of clusters (dendrogram). Example: Document grouping.
Principal Component Analysis (PCA)	Reduces the number of features while keeping the most important information. Example: Image compression.

3. Semi-Supervised Learning

Definition: Uses a small amount of labeled data and a large amount of unlabeled data.

Improve learning accuracy when labeling data is expensive.

Examples:

- Facial recognition (few labeled faces, many unlabeled)
- Web content classification

Algorithms:

- Self-training
- Co-training
- Semi-Supervised SVM

Algorithm	Explanation
Self-Training	The model trains on labeled data, then labels the unlabeled data itself.
Co-Training	Two models teach each other using different sets of features.
Semi-Supervised SVM	Extends SVM to use unlabeled data to improve performance.

4. Reinforcement Learning

The model learns by interacting with the environment and receiving rewards or punishments.

Maximize long-term rewards.

Examples:

- Game playing (e.g., Chess, Go)
- Robot navigation

Algorithms:

- Q-Learning
- Deep Q-Networks (DQN)
- SARSA

Algorithm	Explanation
Q-Learning	Learns the value of actions based on rewards received. Used in game agents.
Deep Q-Network (DQN)	Combines Q-Learning with Deep Neural Networks for complex problems.
SARSA	Similar to Q-Learning but learns based on the actual actions taken, not just optimal ones.

K-NEAREST NEIGHBOUR ALGORITHM

K-Nearest Neighbors (KNN) is a **supervised machine learning algorithm** used for **classification and regression**. It is based on the idea that **similar data points exist close to each other** in the feature space. KNN does not build a model during training; instead, it memorizes the training dataset and makes decisions at the time of prediction based on the closest training examples.

1. Lazy Learner:

KNN is a lazy learner because it does **not train a model**; instead, it stores all the training data and performs computation only during prediction.

2. Distance Metric:

It uses a **distance function** to measure how close test data is to training data. Common metrics include:

- **Euclidean Distance**
- **Manhattan Distance**
- **Minkowski Distance**

3. Value of K:

- 'K' refers to the **number of nearest neighbors** to consider.
- Choosing an appropriate K is crucial:
 - **Low K**: more sensitive to noise.
 - **High K**: may include too many distant points and reduce accuracy.

How KNN Works:

1. **Choose K** (number of neighbors)
2. **Calculate distances** between the test data point and all training data points using a distance metric.
3. **Sort the distances** and find the K closest neighbors.
4. **Classify (for classification)**:
 - Use **majority voting** among the K neighbors to decide the class.
5. **Predict value (for regression)**:
 - Use the **average or weighted average** of the K neighbors.

Example:

Let's say we want to classify whether a fruit is an apple or orange based on its weight and color.

- We have a dataset of labeled fruits.
- For a new unknown fruit, KNN:
 - Calculates distances to all fruits in the dataset.
 - Selects the K nearest ones.
 - Assigns the most frequent label among them (e.g., if 3 out of 5 nearest are apples, it predicts apple).

Advantages of KNN:

- Simple and intuitive
- No training phase (fast training)
- Works well with small datasets
- Non-parametric (no assumptions about data distribution)

Disadvantages of KNN:

- **Slow prediction** time for large datasets
- **Sensitive to irrelevant or scaled features**
- **High memory usage**
- Doesn't work well with **high-dimensional data** (curse of dimensionality)

Applications:

- Handwriting recognition
- Recommender systems
- Fraud detection
- Medical diagnosis

EXERCISE I:

Using KNN Classifier, Choose k as 5

Brightness	Saturation	Class
40	20	Red
50	50	Blue
60	90	Blue
10	25	Red
70	70	Blue
60	10	Red
25	80	Blue
20	35	?

Step 1: Calculate Euclidean Distance

Euclidean Distance Formula:

$$\text{Distance} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

We compute the distance from test point (20, 35) to all training points:

Sample	Brightness	Saturation	Class	Distance to (20, 35)
1	40	20	Red	$\sqrt{((40-20)^2 + (20-35)^2)} = \sqrt{(400 + 225)} = \sqrt{625} = \mathbf{25.00}$
2	50	50	Blue	$\sqrt{(900 + 225)} = \sqrt{1125} \approx \mathbf{33.54}$
3	60	90	Blue	$\sqrt{(1600 + 3025)} = \sqrt{4625} \approx \mathbf{68.03}$
4	10	25	Red	$\sqrt{(100 + 100)} = \sqrt{200} \approx \mathbf{14.14}$
5	70	70	Blue	$\sqrt{(2500 + 1225)} = \sqrt{3725} \approx \mathbf{61.06}$
6	60	10	Red	$\sqrt{(1600 + 625)} = \sqrt{2225} \approx \mathbf{47.17}$
7	25	80	Blue	$\sqrt{(25 + 2025)} = \sqrt{2050} \approx \mathbf{45.28}$

Step 2: Sort by Distance

Top 5 nearest neighbors:

Sample	Class	Distance
4	Red	14.14
1	Red	25.00
2	Blue	33.54
7	Blue	45.28
6	Red	47.17

Step 3: Majority Voting (K = 5)

- **Red:** 3 (Samples 1, 4, 6)
- **Blue:** 2 (Samples 2, 7)

Predicted Class = Red

The predicted class for (Brightness = 20, Saturation = 35) is Red using KNN with K = 5

Solve:

Using KNN Classifier ,Find in which class of people Angelina will lie whose k factor is 3 and age is 5

NAME	AGE	GENDER	CLASS OF SPORTS
Ajay	32	0	Football
Mark	40	0	Neither
Sara	16	1	Cricket
Zaira	34	1	Cricket
Sachin	55	0	Neither
Rahul	40	0	Cricket
Pooja	20	1	Neither
Smith	15	0	Cricket
Laxmi	55	1	Football
Michael	15	0	Football

From the given data-set find $(x,y) = 57,170$ whether belongs to Under or Normal Weights. Choose k as 3

Height (CM)	Weight (KG)	Class
167	51	Underweight
182	62	Normal
176	69	Normal
173	64	Normal
172	65	Normal
174	56	Underweight
169	58	Normal
173	57	Normal
170	55	Normal
170	57	?

NAIVE BAYES CLASSIFIER ALGORITHM

- The **Naive Bayes Classifier** is a **probabilistic machine learning algorithm** used primarily for **classification tasks**.
- The Naive Bayes Classifier is a powerful and efficient classification tool despite its simple assumptions.
- It is called “naive” because it **assumes that all features are independent** of each other given the class
- it performs well in many complex real-world situations, especially in **text classification**.
- It is widely used in many domains, particularly where data is text-heavy or categorical.
- With enhancements like Laplace smoothing and proper preprocessing, it delivers accurate results even in complex tasks.

Bayes' Theorem

Naive Bayes is based on **Bayes' Theorem**, which describes the probability of an event based on prior knowledge.

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

Where:

- $P(C|X)$: Posterior probability of class C given feature vector X
- $P(X|C)$: Likelihood of features given class
- $P(C)$: Prior probability of the class
- $P(X)$: Probability of features

The classifier selects the class C that **maximizes** the posterior probability.

Working Steps

- **Calculate prior probabilities** for each class.
- **Calculate likelihood**: the probability of features for each class.
- **Multiply the probabilities** using Bayes' rule.
- **Classify** the instance into the class with the highest posterior probability.

Assumption of Independence

Naive Bayes assumes that **features are conditionally independent**:

$$P(x_1, x_2, \dots, x_n | C) = P(x_1 | C) \cdot P(x_2 | C) \cdot \dots \cdot P(x_n | C)$$

Though this assumption is rarely true, the algorithm still works well in practice due to its ability to generalize from small datasets.

Types of Naive Bayes Models

Type	Description	Use Case
Gaussian	Assumes features follow a normal distribution	Used when features are continuous
Multinomial	Deals with frequency of words	Text classification
Bernoulli	Binary data (present/absent)	Spam filters, sentiment (positive/negative)

Real-Life Examples

Spam Filtering

- Features: Words like “Free”, “Win”, “Money”
- Classes: Spam, Not Spam
- Naive Bayes calculates the probability that an email belongs to spam or not based on word frequency.

Sentiment Analysis

- Input: Movie review text
- Output: Positive / Negative
- Uses Multinomial Naive Bayes to classify based on words and tone.

Medical Diagnosis

- Input: Symptoms like fever, cough, headache
- Output: Disease prediction (e.g., Flu, COVID)
- Assumes symptom independence given disease.

Advantages

- **Simple and fast**
- Works well on **high-dimensional data**
- Performs better than expected even with **small datasets**
- Excellent for **text classification**

- Requires **less training data**

Disadvantages

- Assumes **independence of features**
- Doesn't perform well with **correlated or dependent features**
- Not ideal for **real probability prediction**
- Performs poorly when there's **zero probability** for unseen features (handled by **Laplace smoothing**)

Common Techniques Used

- **Laplace Smoothing**: To handle zero probabilities
- **TF-IDF with Naive Bayes**: Improves text classification by weighing important words.

Example

We want to predict the Species (M or H) of a new animal based on the following features:

Color = White, Legs = 2, Height = Short, Smelly = Yes

Dataset

Sl. No.	Color	Legs	Height	Smelly	Species
1	White	3	Short	Yes	M
2	Green	2	Tall	No	M
3	Green	3	Short	Yes	M
4	White	3	Short	Yes	M
5	Green	2	Short	No	H
6	White	2	Tall	No	H
7	White	2	Tall	No	H
8	White	2	Short	Yes	H

Step 1: Prior Probabilities

Total records = 8

Species = M → 4 records: $P(M) = 4/8 = 0.5$

Species = H → 4 records: $P(H) = 4/8 = 0.5$

Step 2: Conditional Probabilities

For Species = M:

Feature	Value	Count	Probability
Color	White	2	$2/4 = 0.5$
Legs	2	1	$1/4 = 0.25$
Height	Short	3	$3/4 = 0.75$
Smelly	Yes	3	$3/4 = 0.75$

For Species = H:

Feature	Value	Count	Probability
Color	White	4	$4/4 = 1.0$
Legs	2	4	$4/4 = 1.0$
Height	Short	2	$2/4 = 0.5$
Smelly	Yes	1	$1/4 = 0.25$

Step 3: Apply Naive Bayes Formula

We ignore the denominator since it's the same for both classes.

We compute for both M and H:

$$\blacklozenge P(M | X) \propto P(M) \times P(\text{Color=White} | M) \times P(\text{Legs=2} | M) \times P(\text{Height=Short} | M) \times P(\text{Smelly=Yes} | M)$$

$$P(M|X) = 0.5 \times 0.5 \times 0.25 \times 0.75 \times 0.75 = 0.5 \times 0.5 \times 0.25 \times 0.75 \times 0.75 = 0.0351$$

$$\blacklozenge P(H | X) \propto P(H) \times P(\text{Color=White} | H) \times P(\text{Legs=2} | H) \times P(\text{Height=Short} | H) \times P(\text{Smelly=Yes} | H)$$

$$P(H|X) = 0.5 \times 1.0 \times 1.0 \times 0.5 \times 0.25 = 0.5 \times 1 \times 1 \times 0.5 \times 0.25 = 0.0625$$

Since $P(H|X) > P(M|X)$, the predicted Species is H.

Solve

Problem 1:

Outlook	Temperature	Humidity	Windy	Play
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
Rainy	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Rainy	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Rainy	Mild	High	True	No

X = (outlook = sunny, temperature = cool, humidity = high, wind = true)

Problem II:

Type of Family Structure	Age Group	Income Status	Will They Buy a Car?
Nuclear	Young	Low	Yes
Extended	Old	Low	No
Childless	Middle-aged	Low	No
Childless	Young	Medium	Yes
Single Parent	Middle-aged	Medium	Yes
Childless	Young	Low	No

Given three inputs–Single Parent, Young, and Low—we want to compute the probability of these people buying a car

Problem III:

chills	runny nose	headache	fever	flu?
Y	N	Mild	Y	N
Y	Y	No	N	Y
Y	N	Strong	Y	Y
N	Y	Mild	Y	Y
N	N	No	N	N
N	Y	Strong	Y	Y
N	Y	Strong	N	N
Y	Y	Mild	Y	Y

Now, we will predict whether for the given conditions we will have a flu or not

Chills	Runny Nose	Headache	Fever	Flu?
Y	N	Mild	N	?

LAPLACE ESTIMATOR (LAPLACE SMOOTHING)

The **Laplace Estimator** (also called **Laplace Smoothing** or **add-one smoothing**) is a technique used in **Naive Bayes classifiers** to handle the **problem of zero probability**.

When a feature **value does not appear** in the training data for a class, its probability becomes **zero**, and the entire product of probabilities becomes **zero**, even if other features match well. To avoid this, we **add 1 to the count** of each feature value.

Why do we need it?

Without smoothing:

$$P(\text{feature}|\text{class}) = \frac{\text{Count of feature in class}}{\text{Total count in class}}$$

If count = 0, then $P = 0$, and the product of probabilities becomes 0.

With Laplace smoothing:

$$P(\text{feature}|\text{class}) = \frac{\text{Count} + 1}{\text{Total} + k}$$

Where:

- k = number of possible feature values (vocabulary size)
- This ensures **no probability is ever zero**

Example:

Suppose we have this dataset for a spam classifier:

Word	Spam	Not Spam
Free	3	1
Win	2	0
Hello	0	2

We want to calculate $P(\text{Hello}|\text{Spam})$

- Without Laplace:

$$P(\text{Hello}|\text{Spam}) = \frac{0}{5} = 0$$

- With Laplace ($k = 3$ words in vocabulary):

$$P(\text{Hello}|\text{Spam}) = \frac{0 + 1}{5 + 3} = \frac{1}{8} = 0.125$$

Now the model won't break due to a 0 probability.

Advantages:

- Avoids **zero probability** problems
- Helps the model **generalize** better
- Useful for **text classification** and **small datasets**

$$P(x_i|C) = \frac{\text{Count of } x_i \text{ in class } C + 1}{\text{Total words in class } C + |V|}$$

Formula Summary:

Where:

- x_i = feature (word)
- C = class
- $|V|$ = vocabulary size

Problem 1

Line Speed	Age	Temperature	Motor
Low	Old	High	Bad
Low	Old	Normal	Bad
Normal	New	Normal	Good
Normal	Old	High	Bad
High	Old	High	Bad
High	Old	Normal	Good
Normal	New	Normal	Good
Low	New	High	Bad
Low	New	High	Bad

New Instance: Normal, New, High Determine whether Motor is good or bad?

Problem 2:

Film	Country of Origin	Big Star	Genre	Success
F1	US	Yes	Fiction	T
F2	US	No	Comedy	F
F3	US	Yes	Comedy	T
F4	Europe	No	Comedy	T
F5	Europe	Yes	Fiction	F
F6	Europe	Yes	Romance	F
F7	Rest of world	Yes	Comedy	F
F8	Rest of world	No	Fiction	F
F9	Europe	Yes	Comedy	T
F10	US	Yes	Comedy	T

New Instance: Rest of world, Yes, Comedy